BIKENEXUS

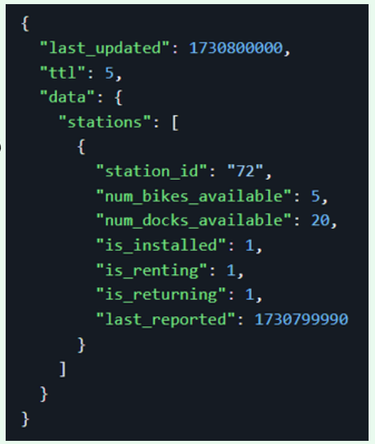
Presentación

Integrantes:

Athina Cappelletti García
David Alejandro Gutiérrez Leal
Emmanuel Álvarez Castrillón
Ginna Alejandra Valencia Macuace
Mariamny Del Valle Ramírez Telles

INGESTIÓN STREAMING

1. API city bike NY



```
{
  "last_updated": 1730800000,
  "ttl": 5,
  "data": {
    "stations": [
      {
        "station_id": "72",
        "num_bikes_available": 5,
        "num_docks_available": 20,
        "is_installed": 1,
        "is_renting": 1,
        "is_returning": 1,
        "last_reported": 1730799990
      }
    ]
  }
}
```

```
{
  "data": {
    "stations": [
      {
        "station_id": "72",
        "name": "W 52 St & 11 Ave",
        "lat": 40.767,
        "lon": -73.993,
        "capacity": 25
      }
    ]
  }
}
```

- EventBridge cada 60s.
- estacion_id como PARTITION KEY.
- At least once.

2. Lambda productora

```
{
  "estacion_id": "72",
  "bikes_disponibles": 5,
  "docks_libres": 20,
  "capacidad": 25,
  "activa": true,
  "timestamp": "2026-11-05T10:30:00Z",
  "fuente_last_reported": 1730799990,
  "ingest_ts": "2026-11-05T10:30:01.123Z"
}
```

3. Kinesis

INGESTIÓN STREAMING

3. Kinesis

4. Spark Structured Streaming

5. Delta lake S3

- Lee desde Kinesis.
- Guarda en Bronze y agrega metadatos.
- Lee desde Bronze en microbatches.
- Deduplica por estacion_id, timestamp para Silver.
- Aparta registros inválidos para silver

CLASIFICACIÓN DE ESTACIONES

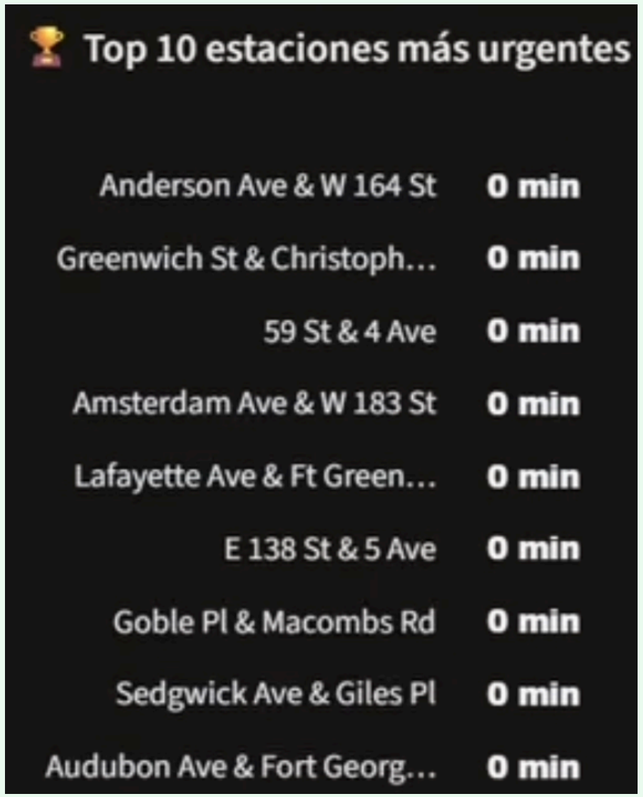
¿Cómo se anticipa que una estación se vaciará?

- Un consumidor en Python (**boto3**) lee los reportes de Kinesis cada 60s.
- Compara cada estación con su reporte anterior (ritmo: bicis que gana o pierde por minuto)
- Tiempo restante = bicis actuales ÷ ritmo (o espacios libres ÷ ritmo)
- Se hace la clasificación de esta forma:
  - Vacía(0 bicis)
  - Llena (0 espacios)
  - En riesgo (< 15min)
  - Normal

ALERTAS Y DISPONIBILIDAD

Alertas Priorizadas para el Operador

- Último estado por estación en DynamoDB (upsert, idempotente)
- Se crea una lista de estaciones ya ordenada por urgencia (en riesgo)
- **Dashboard Streamlit:** KPIs, Top 10 más urgentes y tabla ordenada.
- **AP** (CAP): ante una falla se muestra el último estado conocido y se marcan como desactualizadas las estaciones sin reporte en más de 2 min.



INGESTA BATCH: DE LA FUENTE A SILVER

Lambda → Landing

Descarga con reintentos y backoff, descomprime el zip y sube el CSV sin tocarlo. Cuenta las filas y las anota en manifest.json.

Landing → Bronze

Todo como texto, sin perder registros. Agrega _archivo_origen, _fecha_ingesta y _lote. Delta, particionado por lote.

Bronze → Silver

Tipos, fechas y reglas de calidad. Lo válido va a Silver, el resto a cuarentena con su motivo.

Landing · catálogo
catalogo/fecha=AAAA-MM-DD/catalogo.json

Landing · viajes
viajes/lote=AAAA-MM/*.csv + manifest.json

Disparadores. **EventBridge carga el catálogo cada día. Step Functions corre el lote mensual: Lambda, catálogo, viajes y Gold, con reintentos.**

CATÁLOGO Y CALIDAD

Catálogo

- **Bronze** guarda cada fecha
- **Silver:** una fila vigente por estación, con MERGE
- **Cuarentena:** id nulo, capacidad nula o negativa, coordenadas fuera de rango
- Acepta varios nombres de columna (station_id, id, short_name)

Motivo de cuarentena · viajes	Regla
fecha_invalida	Inicio o fin nulo, o con un formato no reconocido (se prueban 8 patrones)
estacion_nula	Estación de origen o destino vacía
duracion_menor_1_min	Duración menor a 60 s
duracion_mayor_24_h	Duración mayor a 24 h
estacion_origen_desconocida	El origen no existe en el catálogo Silver
estacion_destino_desconocida	El destino no existe en el catálogo Silver

CONTRATOS DE CALIDAD Y CASOS DE BORDE

Contratos de calidad

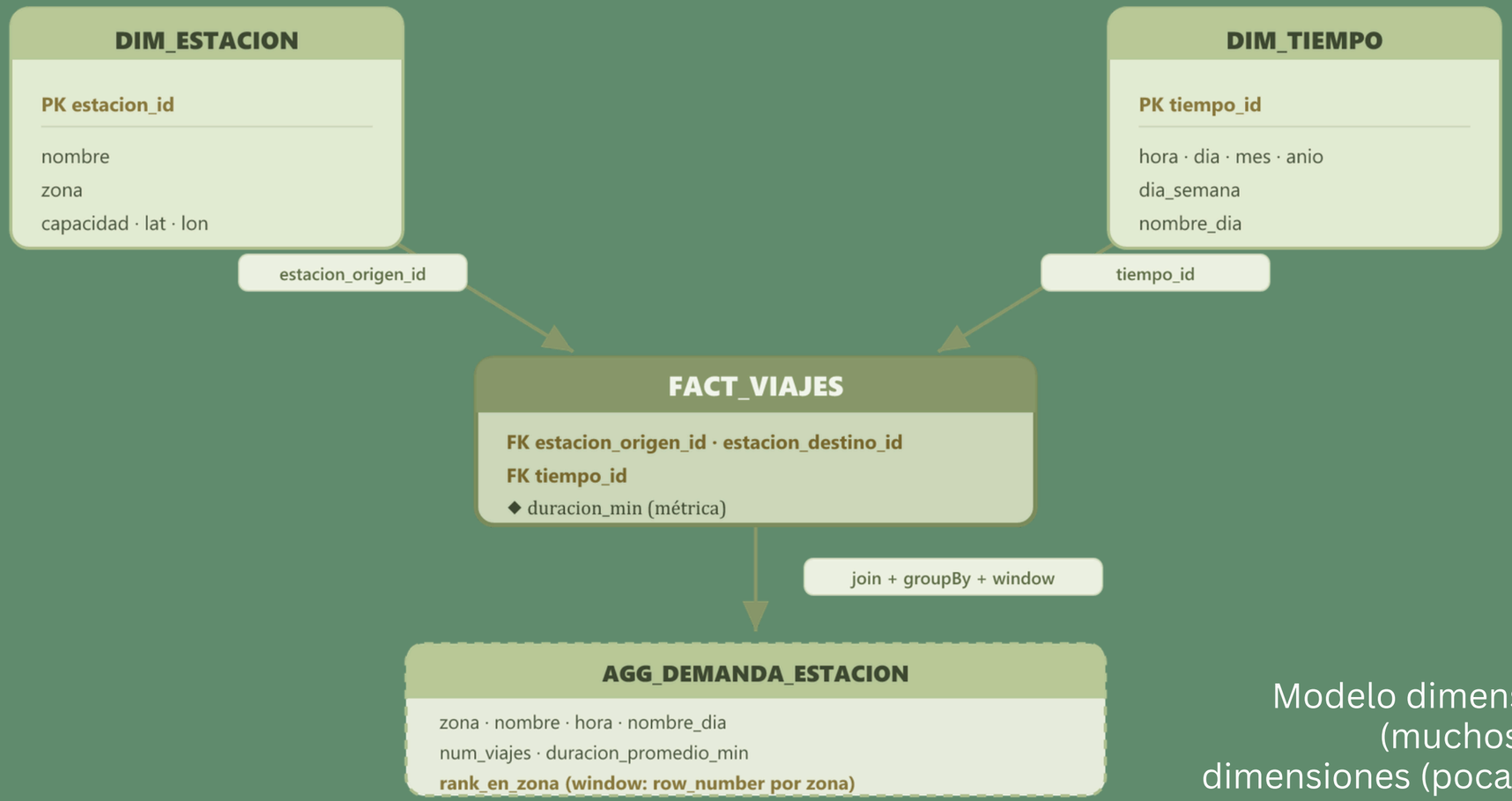
- **Idempotencia:** reprocesar un lote reemplaza solo su partición (replaceWhere)
- **Evolución de esquema:** mergeSchema y columnas nuevas conservadas
- **Conciliación:** Silver + cuarentena = Bronze, o el job falla
- **Trazabilidad (RNF-06):** archivo de origen, fecha de ingesta y lote

Casos de Borde y Riesgos

- **Incompatibilidad de esquemas:** Exige control estricto de tipos o overwriteSchema para evitar interrupción en Delta Lake.
- **Retraso de eventos (Streaming):** Marcas de tiempo desfasadas quedan sujetas a ventanas de marca de agua (watermarking) en la capa analítica.
- **Re-ejecución en fallos de EMR/Kinesis:** Basada en checkpoints persistentes en S3/DynamoDB para preservar idempotencia.
- **Registros severamente corruptos:** Se desvían directamente a cuarentena sin detener el pipeline ni alterar la capa Silver.

CAPA GOLD

STAR SCHEMA



Modelo dimensional = hechos (muchos) separados de dimensiones (pocas, descriptivas)

agg_demanda_estacion:

JOIN + GROUPBY + WINDOW

JOIN → fact × dim_estacion × dim_tiempo (star join)

GROUP BY → viajes y duración promedio por zona / día / hora

WINDOW → row_number() partitionBy("zona") → ranking de estaciones

```
ventana_zona = Window.partitionBy("zona").orderBy(F.col("num_viajes").desc())
resultado = agregado.withColumn("rank_en_zona", F.row_number().over(ventana_zona))
```

GLUE DATA CATALOG + ATHENA

batch_gold.py escribe Delta en S3/gold → Crawler de Glue → Glue Data Catalog (bikenexus_gold) → Athena SQL

Reto:	Athena no lee Delta como Parquet plano
Elegimos:	Crawler de Glue (vs DDL manual + manifiesto)
Por qué:	menos errores, no hay que sincronizar DDL a mano

¿EN QUÉ ZONAS SE CONCENTRA LA DEMANDA Y CUÁLES SON LAS ESTACIONES CLAVE?

Consulta en Athena · pregunta de negocio del E1

```
SELECT zona, nombre, hora, num_viajes, duracion_promedio_min, rank_en_zona
FROM bikenexus_gold.agg_demanda_estacion
WHERE rank_en_zona <= 3
ORDER BY zona, rank_en_zona;
```

zona	nombre	hora	num_viajes	dur_prom_min	rank
Hoboken	Hoboken Terminal - River St & Hudson Pl	17	**112**	6.03	1
	Hoboken Terminal - River St & Hudson Pl	17	**104**	14.67	2
	Hoboken Terminal - River St & Hudson Pl	17	**86**	6.05	3
Grove	Grove St PATH	17	**110**	5.37	1
	Grove St PATH	17	**90**	5.86	2
	Grove St PATH	17	**87**	5.57	3
River	River St & 1 St	18	**59**	5.59	1
	River St & 1 St	18	**47**	6.61	2
	River St & 1 St	18	**39**	6.06	3
Hamilton	Hamilton Park	8	**47**	6.77	1
	Hamilton Park	8	**44**	5.72	2
	Hamilton Park	8	**36**	6.32	3
Newport	Newport PATH	17	**39**	8.87	1
	Newport PATH	17	**34**	8.87	2
	Newport PATH	17	**31**	10.86	3

TRAZABILIDAD + LATENCIA BATCH

TRAZABILIDAD

- *_delta_log* + Time Travel (retención 30 días)
- Metadatos de origen (*meta_shard_id/meta_sequence_number* en streaming · *meta_archivo_origen* en batch)
- Bronze inmutable (fuente de verdad)

LATENCIA

- Streaming: < 1 min (evento → DynamoDB) — *medir_latencia.py*
- Batch: < 24 h → se mide con *stopDate – startDate* de Step Functions, cierra en minutos

INFRAESTRUCTURA

Aseguramiento de la escalabilidad del sistema:
El sistema es capaz de soportar 10 veces la carga normal del sistema

Carga normal = N Estaciones x 60 s, con 2518 estaciones serian unos 42 eventos/s

Picos a soportar: 420 eventos/s

Lo que usamos: 2 shards de Kinesis, cada uno admite hasta 1000 registros/s o 1MB/s de entrada

Lambda: se dispara con rate(1 minute), 256 MB, timeout de 30 s, y publica en lotes de 500 (límite de PutRecords).

Proposito: mantener la latencia de extremo a extremo de las alertas por debajo del minuto

INFRAESTRUCTURA

Tolerancia a fallos:
Productor: reintenta los registros rechazados hasta 5 veces con backoff exponencial (0,1 s, 0,2 s, 0,4 s...).

Kinesis: retención de 24 h para reprocesar tras un fallo.

Spark: checkpointLocation en S3 en bronze_stream.py, silver_stream.py y clasificacion_stream.py, con micro-batches de 30 s.

DynamoDB: escritura por upsert (idempotente).

Duplicados: Silver usa MERGE de Delta con clave (estacion_id, timestamp). Si un micro-batch se reprocesa, no se duplica.

Prueba automatizada:
test_reintento_de_fallidos simula que Kinesis rechaza 2 registros.
Resultado: 0 sin publicar y 2 llamadas, o sea que hizo falta un reintento.